ChatGPT

Evidence→Belief→Recommendation→Trust Architecture and One-Shot Refactor Plan for `hwloser/trade`

Executive summary

This manuscript specifies an Evidence→Belief→Recommendation→Trust (EBRT) architecture for a TradeDB-style daily recommendation system, grounded in the current `hwloser/trade` repository and reinforced by primary research on belief revision, attention mechanisms, residual updates, filtering/denoising, event studies, influence diffusion, and MLOps observability. The key thesis is that a "daily picks" system should be organised around **belief state updates**: raw multi-source evidence is normalised and quality-scored, then used to update a persistent belief state with an **attention-style weighting** and a **residual ("ResNet-like") update**. Recommendations are produced as a deterministic projection of belief + risk + constraints, and trust is maintained via traceability, calibration, data freshness, and run lineage.

The repo already contains strong building blocks for EBRT: (i) a Bronze→Silver→Gold sentiment pipeline with deduplication and incremental enrichment; (ii) explicit noise/reliability features at the article and aggregate level; (iii) an event bus + DAG table for orchestrating jobs; (iv) a web API and pages centred on "Today / Signals / Pipeline"; and (v) a daily evaluation/quality gate that computes calibration metrics (e.g., Brier score binning). These are precisely the "Evidence" and "Trust" halves of EBRT. The missing centre is a first-class **Belief State** layer and a cleaner execution plane: the repository's Python CLI currently exposes many domains and subcommands, and DAG/job triggering still mixes concerns by calling various command routines from job handlers, contributing to "command sprawl" and hard-to-reason pipelines. `trade_py/cli/main.py` explicitly imports and dispatches a wide set of domains (run/status/inspect/backup/data/model/factor/account/event/evaluate/kg/start/web), illustrating this growth pressure. [1]

I recommend a one-shot refactor that (a) replaces "commands everywhere" with a **small Engine API** and a **thin CLI**; (b) collapses many operational tables into a single `ops_*` namespace; (c) introduces canonical EBRT objects (`Evidence`, `BeliefState`, `BeliefTransition`, `RecommendationTrace`) while keeping compatibility views for existing tables; and (d) implement attention and residual updates as a *belief updater* module that can be either symbolic/differentiable (no NN required) or learned end-to-end later.

Background and related work

Belief update has two complementary traditions. The logical tradition (AGM) defines postulates for rational belief change and constructions such as partial-meet contraction/revision, framing belief as a coherent set updated under new information. [2] The statistical tradition treats belief as a latent state estimated from noisy observations; the Kalman filter formalises optimal linear state estimation under Gaussian assumptions and explicitly models uncertainty and update gain. [3] EBRT intentionally borrows *structure* from both: we model persistent belief state and rational conflict handling (AGM), while

implementing update rules as state-space style residual updates with explicit noise-aware gains (Kalman-inspired).

Transformers introduced attention as a normalised compatibility weighting that determines how strongly each piece of context contributes to the next representation. [4] In EBRT, "attention" is not necessarily a neural layer; it can be a **symbolic attention** (softmax over evidence scores) or an embedding-based similarity. Residual networks showed that learning (and updating) is stabilised by expressing change as a residual added to an identity path. [5] EBRT applies this idea directly: belief updates are written as *old belief + gated delta*, creating inertia and reducing overreaction to noise.

Market event studies provide a measurement discipline for "catalysts": abnormal returns around events are estimated via market models and robust test statistics; Brown & Warner's methodology and MacKinlay's survey are canonical references. [6] This literature motivates EBRT's separation between **event evidence** (what happened), **belief** (expected impact), and **evaluation** (did the impact materialise, how often, under what conditions).

Influence/catalyst detection extends beyond events to "who caused amplification". Influence maximisation research (independent cascade / linear threshold models) formalises diffusion and the importance of seed nodes, giving a principled lens for "big-V posts can move markets" (with the obvious caveat that noise and manipulation exist). [7] EBRT uses influence modelling to weight evidence by source credibility and propagation patterns rather than raw volume.

Denoising is a primary theme in representation learning. Denoising autoencoders learn representations robust to corrupted inputs, and stacking them provides a principled denoising objective. [8] For EBRT, denoising is both (i) *pre-attention noise reduction* (filter low-quality evidence), and (ii) *attention gating* (prevent noisy evidence from receiving high weight).

Operational trust in ML systems is shaped by MLOps observability and evaluation standards. The "ML Test Score" provides a rubric of production readiness, emphasising monitoring, testing, and data/feature validation. [9] Model Cards formalise model documentation for transparency, limitations, and evaluation context—directly aligned with EBRT's Trust contract. [10] Finally, calibration metrics such as the Brier score (probabilistic forecast accuracy) ground EBRT's "confidence" claims in measurable reliability. [11] Drift detection via distributional tests like MMD gives a practical mechanism for "belief reliability decay" when the data distribution shifts. [12]

Repository scan and problem diagnosis

High-level structure and intent

The repo positions itself as a unified CLI and UI system: the README states the unified entry `./trade`, supporting a C++ CLI build and Python workflows, and a Streamlit-based UI entry (`trade_py/ui/ui.py`). [13] The authoritative refactor plan in `docs/claude-latest-plan.md` documents a deliberate split: keep `engine/` as high-performance compute/storage, and move ingestion/backfill/account metadata to Python (`trade_py/`). It also includes an explicit directory blueprint for `engine/`, `trade_py/`, `trade_web/`, `config/`, `docs/`, etc. [14] This confirms that the repository is already migrating towards the "data+ML+ops in Python" architecture that EBRT assumes.

Evidence pipeline is already present, including denoising hooks

The sentiment pipeline is explicitly organised as Bronze→Silver→Gold, with deduplication and incremental processing. `ingest.py` ingests raw articles and deduplicates by `content_hash`, persisting ingestion runs to a pipeline state DB. [15] `enrich.py` runs enrichment for only new hashes and writes Silver parquet; it uses cached enriched hashes stored in the pipeline DB for incremental runs. [16] `aggregate.py` then computes Gold daily factors (net sentiment, velocity, shock, counts) and **computes a credibility score** `signal_strength` that explicitly combines volume, source diversity, label consensus, mean confidence, and a noise penalty. [17]

Crucially, the repo *does contain denoising algorithms*, but they are implemented as *quality scores and penalties*, not as a standalone "denoise module". `base_factors.py` computes a `noise_score` from novelty, clickbait markers, entity density, and title-body overlap, and subtracts it from event magnitude and confidence. [18] The enrichment logic (`enricher.py`) performs **reliability gating** between LLM and deterministic extraction: it uses LLM output only if LLM confidence exceeds a threshold (≥0.6); otherwise it falls back to the base extractor. [19] These are concrete EBRT "Evidence quality" and "Trust gating" primitives already in production.

Execution plane exists but is entangled with CLI and "too many commands"

The CLI dispatch imports many domains. [1] `trade_py/cli/event.py` provides DAG inspection, job runs, triggering events, syncing and backfilling events; it is effectively an operations console in command form. [20] `trade_py/cli/run.py` adds "workflow targets" that map to topics and jobs, further layering orchestration into CLI space. [21]

At runtime, an in-process event bus bootstraps subscriptions from a `pipeline_dag` table; it publishes events and invokes job handlers. [22] Jobs are registered in `trade_py/jobs/__init__.py`, but at least one job explicitly calls a CLI entry (`trade_py.cli._sentiment.main`) from within the job function, illustrating the "DAG calls command" complaint. [23] This design works, but it blurs domain boundaries and makes further iteration likely to increase command/job proliferation.

Observability and Trust are already unusually strong for a personal research system

The web backend (`trade_web/backend/app.py`) already exposes endpoints for DAG state, runtime streams, workflows, run records, data health, and "today-page / signals-page / kline". [24] The repo also defines a daily evaluation service that computes source health, event yield, model evaluation metrics (including calibration bins and Brier score for a risk model), fingerprints for caching, and an overall quality gate ("operational status" vs "research status"). [25] This is directly aligned with EBRT's "Trust" layer: the system already knows how to say "pipeline ready vs pipeline degraded" and can quantify model reliability.

EBRT architecture specification

Seven-layer EBRT design

The EBRT architecture reframes the system around **a persistent Belief State** that is updated from Evidence and projected into Recommendations, with Trust providing observability, calibration, and auditability.

```
flowchart TB
  subgraph L1[Source layer]
    S1[Market data\n(OHLCV, flows, fundamentals)]
    S2[Articles/News/Social]
    S3[Reference\n(symbols, sectors)]
    S4[Calendar/Planned events]
  end

  subgraph L2[Ingestion & normalisation]
    I1[Batch ingest\n(daily bars, fundamentals)]
    I2[Stream ingest\n(article polling)]
    I3[Canonicalisation\nIDs, hashes, timestamps]
  end

  subgraph L3[Evidence layer]
    E1[Bronze: raw records]
    E2[Silver: extracted semantics\n(sentiment, event_type,\nentities,
confidence,\nnoise_score)]
    E3[Gold: aggregated evidence\n(per symbol/day)\n(signal_strength,
\nvelocity, shocks)]
    E4[Influence signals\n(source reputation,\npropagation graph)]
  end

  subgraph L4[Belief layer]
    B1[BeliefState store\n(per symbol + market)]
    B2[Attention scorer\nrelevance × reliability × novelty]
    B3[Residual belief update\ninertia + decay + conflict handling]
    B4[BeliefTransition log\n(why belief changed)]
  end

  subgraph L5[Decision layer]
    D1[Candidate generation\n(recall)]
    D2[Ranking\n(coarse→fine)]
    D3[Risk/constraints\n(sector diversity,\nexposure, drawdown)]
    D4[Recommendation\n+ trace]
  end

  subgraph L6[Trust & operations]
    T1[Quality gate\n(data coverage,\nmodel eval,\ncalibration)]
    T2[Lineage & run records\n(DAG, job_runs,\nworkflows)]
    T3[Freshness status\n(sync_state, gaps)]
    T4[Alerts & anomaly\n(drift/MMD,\nspikes, conflicts)]
  end

  subgraph L7[Product layer]
    P1[Web panels\n(Today/Picks/Pipeline)]
    P2[API]
    P3[Exports\n(journal, backtest)]
  end
```

```
S1-->I1-->E1
S2-->I2-->E1
S3-->I3
E1-->E2-->E3
E2-->E4
E3-->B2-->B3-->B1
E4-->B2
B1-->D1-->D2-->D3-->D4-->P1
B4-->D4
T1-->P1
T2-->P1
T3-->P1
T4-->P1
D4-->T2
E1-->T3
E2-->T2
```

This architecture matches existing repo elements: Bronze/Silver/Gold already exist (`ingest.py`, `enrich.py`, `aggregate.py`). [15] [16] [17] Trust/ops primitives exist via `event_log`, `job_runs`, runtime streaming endpoints, and daily evaluation/quality gate. [24] [25]

Where "attention" lives, and why the core algorithm need not be a neural network

In EBRT, **attention is the Belief layer's weighting mechanism**, not necessarily a deep model. "Query" is the current belief context (symbol state, market regime), "keys" are evidence items (events, articles, aggregated signals), and attention weights determine how much each evidence item moves belief.

This answers the user's conceptual questions:

- **Is the core algorithm a neural network?** Not necessarily. The core is the *Belief updater* (attention + residual update). It can be a symbolic scoring model with softmax weights, a probabilistic filter, or a learned NN. EBRT is a system contract, not a single model class.
- **Is attention like IC similarity?** It can be. If you define compatibility as correlation-to-outcome (IC) or embedding similarity, attention becomes "normalised similarity weights".

The repository already uses a "pre-attention gate": Silver and Gold compute reliability (`confidence`, `noise_score`, `signal_strength`) and only high-confidence LLM extraction is adopted. [18] [19] [17] EBRT formalises this into a unified attention contract.

Residual belief update design

I recommend a residual update inspired by ResNet skip connections: let belief change be a bounded delta added to the previous state, improving stability under streaming noise. This is conceptually aligned with residual learning easing optimisation and preventing degradation in deep systems. [5]

Let $b_t \in \mathbb{R}^d$ be a belief vector for a symbol at day t. Evidence items $e_{t,i}$ generate proposed deltas $\Delta_{t,i}$. Attention weights $w_{t,i}$ sum to 1. A residual update:

$$b_{t+1} = \underbrace{(1-\lambda)\, b_t}_{\text{inertia/decay}} + \underbrace{\eta_t \sum_i w_{t,i}\, \Delta_{t,i}}_{\text{evidence-driven residual}} .$$

- $\lambda \in [0,1]$ is time decay (belief forgets slowly).
- $\eta_t \in [0,1]$ is an update gain ("Kalman gain analogue"), reduced when trust is low or drift is detected. [26]

This provides "iteration" without backprop: each day's belief is updated deterministically. If you later want end-to-end learning, you can backprop through this recurrence (Transformer-style attention + residual), but it is optional.

Denoising strategies in EBRT and why attention does not eliminate the need to denoise

Attention *does not magically remove noise*. Transformers attend to salient tokens—but without reliability priors, they can attend to spurious cues. [4] In market data, where adversarial or clickbait signals exist, attention can amplify noise unless you constrain attention logits with source reliability, novelty penalties, and conflict checks. The repo already does this implicitly: `noise_score` penalises clickbait and low entity density; Gold `signal_strength` penalises noise and rewards cross-source confirmation. [18] [17]

I recommend four denoising layers:

1. **Per-source hygiene**: deduplication, rate limiting, and feed health scoring. `PipelineDb` tracks ingestion runs; `feed_scorer.py` computes coverage/uniqueness/reliability to score feeds. [27] [28]
2. **Per-evidence noise scoring**: compute noise features (novelty, clickbait, entity density, title-body overlap). This is already implemented as `noise_score`. [18]
3. **Attention gating**: integrate reliability into attention logits so noise seldom receives weight (see Algorithms section).
4. **Optional learned denoising**: if you embed raw articles, you can add a denoising objective (DAE) for robustness; stacked denoising autoencoders provide a canonical approach. [8]

In practice: **yes**, different sources benefit from different denoise policies because their noise distributions differ (RSS vs GDELT vs social). The repo's source health evaluation already measures duplicate rate and empty-day rate by source. [25] EBRT formalises these as per-source priors that directly influence attention.

Influence and catalyst detection

For "big-V posts causing shocks", EBRT introduces an `InfluenceSignal` object. The goal is not to "believe big-V", but to quantify *whether downstream confirmation* appears and whether historical impact exists.

Two primary toolkits: - **Event study**: treat the post as an event and track abnormal returns and post-event drift. [6]
- **Diffusion/influence propagation**: model how mentions propagate across sources and how quickly independent confirmations appear; influence maximisation research provides principled diffusion models. [7]

In EBRT, influence is a multiplicative term in attention logits (credible influence boosts weight; suspicious influence without confirmation is penalised).

Data model and algorithms

Core EBRT schemas

Below are SQL-like schemas for EBRT canonical objects. I recommend implementing them in `trade.db` (SQLite) for metadata/ops, and Parquet/DuckDB for high-volume facts (evidence bodies, embeddings). This matches the repo's existing split: parquet for sentiment tiers and SQLite for orchestrations and evaluation. [24] [25]

```
-- Atomic extracted article-level semantics (Silver-like)
CREATE TABLE IF NOT EXISTS ArticleEvent (
  article_id       TEXT PRIMARY KEY,          -- content_hash
  published_at     TEXT NOT NULL,
  source_id        TEXT NOT NULL,
  feed_name        TEXT,
  url              TEXT,
  title            TEXT,
  body_ref         TEXT,                       -- pointer to parquet row /
blob
  symbol           TEXT NOT NULL,              -- one row per (article,
symbol)
  event_type       TEXT,
  event_magnitude  REAL,
  sentiment_score  REAL,
  sentiment_label  TEXT,
  policy_signal    INTEGER,
  entity_density   REAL,
  novelty_score    REAL,
  noise_score      REAL,
  extractor        TEXT NOT NULL,              -- base|llm|hybrid
  extractor_conf   REAL NOT NULL,
  created_at       TEXT DEFAULT CURRENT_TIMESTAMP
);

-- Influence signals (e.g., big-V, platform accounts, source reputation)
CREATE TABLE IF NOT EXISTS InfluenceSignal (
  influence_id     TEXT PRIMARY KEY,
  source_id        TEXT NOT NULL,
  actor_id         TEXT,                       -- influencer identifier if
available
  platform         TEXT,                       -- twitter/weibo/...
  published_at     TEXT NOT NULL,
  topic_tags       TEXT,
  reach_estimate   REAL,                       -- followers/views proxy
  reputation_score REAL,                       -- long-run reliability
  manipulation_risk REAL,                      -- bot/spam likelihood
  cross_confirm_1h REAL,                       -- confirmations within 1h
```

```
window
  cross_confirm_24h REAL,
  created_at       TEXT DEFAULT CURRENT_TIMESTAMP
);

-- Evidence: normalised units used in belief update (symbol/day granularity)
CREATE TABLE IF NOT EXISTS Evidence (
  evidence_id      TEXT PRIMARY KEY,
  as_of_date       TEXT NOT NULL,
  symbol           TEXT NOT NULL,
  evidence_type    TEXT NOT NULL,              -- sentiment_gold|
market_event|flow_spike|...
  payload_ref      TEXT NOT NULL,              -- parquet row / json pointer
  strength         REAL NOT NULL,              -- e.g., signal_strength
  direction        REAL NOT NULL,              -- [-1, +1]
  reliability      REAL NOT NULL,              -- [0, 1]
  novelty          REAL NOT NULL,              -- [0, 1]
  noise_penalty    REAL NOT NULL,              -- [0, 1]
  influence_boost  REAL NOT NULL,              -- [0, 1]
  created_at       TEXT DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(as_of_date, symbol, evidence_type, payload_ref)
);

-- BeliefState: persistent “current view” of symbol/market
CREATE TABLE IF NOT EXISTS BeliefState (
  as_of_date       TEXT NOT NULL,
  symbol           TEXT NOT NULL,
  belief_vec_json  TEXT NOT NULL,              -- JSON array length d
  belief_version   TEXT NOT NULL,
  confidence       REAL NOT NULL,              -- calibrated belief
confidence
  uncertainty      REAL NOT NULL,              -- state uncertainty proxy
  updated_at       TEXT DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY(as_of_date, symbol)
);

-- AttentionScore: explanation + audit for weighting
CREATE TABLE IF NOT EXISTS AttentionScore (
  attention_id     TEXT PRIMARY KEY,
  as_of_date       TEXT NOT NULL,
  symbol           TEXT NOT NULL,
  evidence_id      TEXT NOT NULL REFERENCES Evidence(evidence_id),
  logit            REAL NOT NULL,
  weight           REAL NOT NULL,
  factors_json     TEXT NOT NULL,              -- breakdown: reliability,
novelty, similarity, etc
  created_at       TEXT DEFAULT CURRENT_TIMESTAMP
);

-- BeliefTransition: residual update record (“what changed and why”)
CREATE TABLE IF NOT EXISTS BeliefTransition (
```

```
  transition_id    TEXT PRIMARY KEY,
  symbol           TEXT NOT NULL,
  t_date           TEXT NOT NULL,
  t1_date          TEXT NOT NULL,
  prev_belief_ref  TEXT NOT NULL,              -- pointer or snapshot id
  next_belief_ref  TEXT NOT NULL,
  delta_vec_json   TEXT NOT NULL,
  decay_lambda     REAL NOT NULL,
  gain_eta         REAL NOT NULL,
  conflict_score   REAL NOT NULL,
  attention_set_id TEXT NOT NULL,              -- group of AttentionScore
rows
  created_at       TEXT DEFAULT CURRENT_TIMESTAMP
);

-- Recommendation: the produced action for the day
CREATE TABLE IF NOT EXISTS Recommendation (
  rec_id           TEXT PRIMARY KEY,
  as_of_date       TEXT NOT NULL,
  symbol           TEXT NOT NULL,
  action           TEXT NOT NULL,              -- buy|sell|watch|avoid
  conviction       TEXT NOT NULL,              -- low|mid|high
  score            REAL NOT NULL,              -- final ranking score
  risk             REAL NOT NULL,
  horizon_days     INTEGER NOT NULL,
  reasons_json     TEXT NOT NULL,              -- short bullet reasons
  created_at       TEXT DEFAULT CURRENT_TIMESTAMP,
  UNIQUE(as_of_date, symbol)
);

-- Quality report (Trust): summary of readiness
CREATE TABLE IF NOT EXISTS QualityReport (
  eval_date        TEXT PRIMARY KEY,
  operational_status TEXT NOT NULL,            -- ok|degraded|blocked
  research_status  TEXT NOT NULL,              -- ok|partial|blocked
  brier_score      REAL,
  calibration_json TEXT,
  drift_mmd        REAL,
  reasons_json     TEXT NOT NULL,
  metrics_json     TEXT NOT NULL,
  created_at       TEXT DEFAULT CURRENT_TIMESTAMP
);

-- Run records (Ops): unify job_runs/workflows/etc logically
CREATE TABLE IF NOT EXISTS RunRecord (
  run_id           INTEGER PRIMARY KEY AUTOINCREMENT,
  run_kind         TEXT NOT NULL,              -- job|workflow|backfill
  name             TEXT NOT NULL,
  stage            TEXT,
  status           TEXT NOT NULL,              -- ok|error|running|skipped
  started_at       TEXT NOT NULL,
```

```
  completed_at     TEXT,
  elapsed_ms       INTEGER,
  trigger_ref      TEXT,                       -- event_id or schedule id
  inputs_json      TEXT,
  outputs_json     TEXT,
  error            TEXT
);

-- Freshness snapshot (Trust): per dataset
CREATE TABLE IF NOT EXISTS FreshnessStatus (
  as_of_date       TEXT NOT NULL,
  dataset          TEXT NOT NULL,              -- kline|fund_flow|
sentiment_gold|...
  freshness_date   TEXT,
  lag_days         INTEGER,
  coverage_pct     REAL,
  status           TEXT NOT NULL,              -- ok|partial|error
  details_json     TEXT,
  updated_at       TEXT DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY(as_of_date, dataset)
);

-- RecommendationTrace: end-to-end evidence-to-action audit
CREATE TABLE IF NOT EXISTS RecommendationTrace (
  trace_id         TEXT PRIMARY KEY,
  as_of_date       TEXT NOT NULL,
  symbol           TEXT NOT NULL,
  rec_id           TEXT NOT NULL REFERENCES Recommendation(rec_id),
  belief_transition_id TEXT,
  top_evidence_json TEXT NOT NULL,             -- evidence ids with weights
  model_versions_json TEXT,                    -- inference model ids
  data_fingerprint  TEXT NOT NULL,             -- hash of key datasets
  created_at        TEXT DEFAULT CURRENT_TIMESTAMP
);
```

These schemas intentionally correspond to existing repo concepts: - Evidence: Gold sentiment fields and `signal_strength` already exist in aggregation. [17]
- Trust: daily evaluation computes Brier and calibration bins; quality should be materialised similarly. [25]
- Ops: the repo already uses `job_runs`, `event_log`, `pipeline_dag`, UI snapshots and SSE streams. [24]

Symbolic attention logits and residual update rules

Let current context ("query") for symbol s at day t be $q_{t,s}$ derived from belief vector $b_{t,s}$ plus optional market regime and sector embeddings. Each evidence item i has features:

- r_i: reliability (from source health, extractor confidence, historical IC)
- n_i: novelty (inverse repetition)
- ν_i: noise penalty (clickbait/no-entity/low-overlap)

- u_i: influence boost (credible influencer with fast confirmations)
- σ_i: similarity (embedding dot product or rule-based match with current belief topic)
- d_i: direction (signed impact)
- m_i: magnitude/strength

A practical attention logit:

$$\ell_{t,s,i} = \alpha\,\sigma_i + \beta\,\log(1+m_i) + \gamma\,\log(\epsilon+r_i) + \delta\,\log(\epsilon+n_i) - \kappa\,\log(\epsilon+\nu_i) + \xi\,\log(\epsilon+u_i) - \rho\,\mathrm{conflict}(i, b_{t,s}).$$

Weights:

$$w_{t,s,i} = \frac{\exp(\ell_{t,s,i}/\tau)}{\sum_j \exp(\ell_{t,s,j}/\tau)}.$$

- Temperature τ controls "how peaky"; higher τ reduces sensitivity to spiky evidence.
- `conflict()` penalises evidence that contradicts strong existing belief unless it is highly reliable (AGM-inspired conservative revision). [2]

Residual update:

$$b_{t+1,s} = (1-\lambda)\,b_{t,s} + \eta_{t,s}\sum_i w_{t,s,i}\,\Delta(e_{t,s,i}).$$

Where $\Delta(\cdot)$ maps evidence into belief-space deltas, often as:

- deterministic rule templates for early system verification; or
- a learned linear map; or
- an MLP (if you want NN).

Gain schedule (noise-aware):

\eta_{t,s} = \text{clip}\Big(\eta_0 \cdot \text{TrustGate}_t \cdot (1-\text{Drift}_t)\cdot \overline{r}\Big, 0, 1\Big).

- `TrustGate` comes from QualityReport: if operational status is degraded/blocked, reduce updates and/or block daily recommendations (the repo already computes such gates). [25]
- `Drift` can be MMD between recent evidence distributions and a baseline window. [12]

Conflict handling and belief inertia

I recommend a two-stage conflict handler (AGM-style conservatism + practical heuristics):

1. **Intra-day evidence conflict**: if evidence directions disagree, reduce net delta magnitude unless cross-source confirmation is high; the Gold aggregation already approximates this by consensus and diversity in `signal_strength`. [17]
2. **Belief vs evidence conflict**: if belief strongly bullish but new evidence is bearish, require higher reliability to reverse; implement with an additional penalty term in logits and/or an asymmetric gain.

A workable heuristic for conflict penalty:

$$\mathrm{conflict}(i, b) = \max(0, -\mathrm{sign}(d_i)\cdot \mathrm{proj}(b))$$

where $\mathrm{proj}(b)$ is a scalar belief direction for the relevant dimension (e.g., “policy easing” vs “tightening”).

Learned updates and backpropagation

The repo’s web-ui plan notes that LightGBM does not support true online learning and recommends weekly retraining with real-time feature updates and inference. [29] This is directionally correct for most deployments: gradient boosting is typically trained offline; you can continue training from an existing model, but it is not the same as stable online learning across non-stationary markets. LightGBM’s API does support **continue training** via `init_model` and controlling `keep_training_booster`, which can be used for warm-starting periodic retrains. [30]

Where does backprop fit?

- If you choose NN-based belief update (attention + residual + MLP), you can backpropagate through time across daily steps. This is optional.
- If you embed articles using a pretrained encoder, the encoder itself was trained with backprop, but your system can treat embeddings as fixed features.
- If you introduce denoising autoencoders for article representations, that module is trained with backprop on reconstruction/denoising objectives. [8]
- Even if you keep everything “symbolic”, you can still **learn scalar weights** $(\alpha, \beta, \gamma, \dots)$ by minimising backtest loss; this is “backprop-like” optimisation, but on a tiny parameter set and easy to interpret.

Embedding articles and embedding factors

Yes, you can directly embed articles and let a model learn from embeddings plus sentiment factors. In EBRT terms this becomes: ArticleEvent → Evidence with `payload_ref` pointing to an embedding vector. The key constraints are (i) concept drift, (ii) sample efficiency, and (iii) observability.

- Embeddings help with semantic generalisation (e.g., recognising a policy tightening narrative across different wordings).
- Sentiment factors remain useful because embeddings alone can blur polarity; the repo already computes structured sentiment and policy signals. [18]

Do factors need embedding? Generally, **scalar factors do not need embedding**; they are already numerical features. What can be embedded usefully are: - **factor identities** (if you build a meta-learner over factor graphs), or - **factor interactions** (learned representations of feature sets), or - **categorical contexts** (sector, regime, event_type) as embeddings.

For early correctness/completeness validation, keep factors explicit and interpretable; introduce embeddings at the Evidence layer and/or as auxiliary features.

One-shot refactor plan for code and tables

Target refactor principle

The repo’s `docs/claude-latest-plan.md` already embraces “structure and responsibilities should converge” and aims at a unified Python CLI. [14] The remaining pain points the user identified—

commands too many, **DAG awkward and calls commands**, and **too many tables**—can be addressed by a single "EBRT consolidation" refactor:

1. **Make "Engine API" the only executable surface** for pipelines. CLI becomes thin wrappers.
2. **Make DAG nodes call Engine functions**, never CLI commands.
3. **Move all operational tables into a small ops schema**, and treat many "reporting tables" as materialised views/snapshots rather than permanent independent tables.
4. **Introduce BeliefState and RecommendationTrace** as first-class objects; keep compatibility by writing or generating existing `signals` outputs from EBRT recommendation outputs.

Proposed package layout

I recommend introducing the following packages under `trade_py/`:

- `trade_py/evidence/` (ingest, enrich, aggregate, influence, embedding)
- `trade_py/belief/` (belief state store, updater, attention, transitions)
- `trade_py/decision/` (recall, rank, constraints, recommendation, explain)
- `trade_py/ops/` (run records, dag, scheduler, quality gate, freshness, lineage)
- `trade_py/engine/` (public orchestration API: `run_node`, `run_daily`, `update_belief`, `produce_picks`)

This aligns with the current CLI dispatch domains and allows you to kill command sprawl by making CLI merely call `trade_py.engine.*`. The current CLI explicitly lists domains and imports many modules. [1]

Mapping existing modules to EBRT packages and engine roles

The mapping below uses only files confirmed in the repo scan and is designed to be executed as a one-shot move with compatibility shims.

Current module/file	What it currently does	EBRT package	Engine role after refactor
`trade_py/data/pipeline/ingest.py`	Bronze ingest + dedup by `content_hash`	`trade_py/evidence/ingest.py`	`engine.ingest_articles()` [15]
`trade_py/intelligence/enricher.py`	LLM/base extraction, confidence gate, symbols mapping	`trade_py/evidence/extract.py`	`engine.enrich_articles()` [19]
`trade_py/data/pipeline/enrich.py`	Incremental silver build using cached hashes	`trade_py/evidence/silver.py`	`engine.build_silver()` [16]
`trade_py/data/pipeline/aggregate.py`	Gold aggregation + `signal_strength` and noise penalty	`trade_py/evidence/gold.py`	`engine.build_gold()` [17]
`trade_py/intelligence/base_factors.py`	Deterministic extraction + `noise_score`	`trade_py/evidence/quality.py`	shared by extractors [18]

Current module/file	What it currently does	EBRT package	Engine role after refactor
`trade_py/bus/__init__.py`	EventBus + DAG bootstrap + dispatch	`trade_py/ops/bus.py`	`engine.dispatch_event()` [22]
`trade_py/jobs/__init__.py`	Job registry; includes job calling CLI routine	`trade_py/ops/jobs.py`	Jobs call engine, not CLI [23]
`trade_py/data/access/gateway.py`	DataGateway, gap fill, repair runs	`trade_py/ops/freshness.py` + `trade_py/evidence/market.py`	`engine.ensure_dataset()` [31]
`trade_py/evaluation/service.py`	Source/model/event eval, quality gate, Brier/calibration	`trade_py/ops/quality.py`	`engine.evaluate_daily()` [25]
`trade_py/signals/window_scorer.py`	Window score 0–100, uses sentiment strength blending	`trade_py/decision/features.py`	becomes evidence→belief feature, not final signal [32]
`trade_web/backend/app.py`	Web API, DAG control, snapshots, today/signals pages	`trade_py/ops/api.py` (or keep)	call engine endpoints not CLI [24]
`trade_web/backend/inference.py`	Loads active models from registry, predicts using factors	`trade_py/decision/model_infer.py`	`engine.predict()` [33]

Command consolidation strategy

The system currently has both "domain commands" (`trade <domain> ...`) and operational commands (`trade event ...`, `trade run ...`). [1] [20] The refactor should reduce to three end-user command families:

1. `trade daily` – run the daily EBRT (ingest→evidence→belief→recommendation→trust report).
2. `trade ops` – inspect status, run DAG node, backfill, replay (the operator toolbox).
3. `trade dev` – debugging utilities (inspect parquet, test extractors, etc).

Everything else becomes subcommands or internal functions. `trade_py/cli/run.py` already tries to provide a unified trigger interface; after refactor it should call `engine.run_target()` directly rather than mapping to event CLI calls. [21]

Table consolidation strategy

The repo already seeds and manages many operational tables (`pipeline_dag`, `event_log`, `job_runs`, sync state, UI snapshots). [34] [24] The most effective reduction is not "delete tables blindly", but:

- **Collapse operational tables into a minimal ops set**: `ops_run_records`, `ops_dag`, `ops_events`, `ops_freshness`, `ops_quality`, `ops_snapshots`.
- **Treat many "derived" analytics as views or parquet** (e.g., rather than storing every intermediate ranking table).
- **Keep compatibility views** so existing UI endpoints continue to function during migration.

A practical "migration mapping" (conceptual) is:

Existing (logical) tables/ features	Replace with	Notes
`job_runs`, workflow traces, some event log fields	`RunRecord`	unify run lineage; keep old table as view for UI
`sync_state` + `data_gaps`/repair logs	`FreshnessStatus` + `RunRecord`	DataGateway already logs gaps/ repair behaviour [31]
`signals` as cache for scores	`Recommendation` + derived "signals view"	preserve API contract; compute from recommendations
scattered "reason/explain" structures	`RecommendationTrace`	end-to-end evidence→belief→action audit

This aligns with the repo UI direction: the web UI plan explicitly seeks "recommendations with delta and explanation" and "pipeline nodes show execution time and data time." [29]

Alternatives comparison

Design choice	Option A	Option B	Recommendation
Belief update	Symbolic attention + residual update	End-to-end Transformer time-series	Start with A for correctness/ explainability; add B later if data scale supports [35]
Streaming integration	Streaming only updates evidence snapshots	Fully streaming end-to-end DAG	Use evidence streaming + daily belief/ reco batch; matches repo plan's mixed mode and reduces complexity [29]
Explainability	Rule templates + trace tables	LLM-only explanation	Use rule+trace as canonical; LLM summarises bounded context (Model Cards mindset) [36]

Observability, evaluation and rollout roadmap

Three web panels for EBRT observability

The repo already exposes endpoints for “today-page / signals-page / dag/runtime” and streams. [24] EBRT formalises them as three panels with explicit trust affordances.

Panel: Today (system readiness + market snapshot + top picks)

```
┌──────────────────────────────────────────────────────────────┐
│ TODAY  (as_of: 2026-03-18)                                   │
├──────────────────────────────────────────────────────────────┤
│ TRUST GATE:  operational=OK   research=PARTIAL   drift=MID   │
│ Reasons:  - labeled_propagation_ratio below threshold        │
│           - model_rank_ic_5d slightly weak                   │
├──────────────────────────────────────────────────────────────┤
│ DATA FRESHNESS                                               │
│  Kline: 2026-03-17  lag=1d  cov=96%  [OK]                    │
│  Sentiment Gold: 2026-03-17 lag=1d  rows=5,102 [OK]          │
│  Fund Flow: 2026-03-17 lag=1d  cov=88%  [PARTIAL]            │
├──────────────────────────────────────────────────────────────┤
│ TOP PICKS (with delta)                                       │
│  1) 600000.SH  ★★★  BUY   score 0.82  [NEW]                  │
│     Why: Policy easing evidence, sentiment improving, risk low│
│  2) 000001.SZ  ★★☆  WATCH score 0.71  [CONT]                 │
│     Why: Window clean, flows stabilising                     │
│  ...                                                         │
└──────────────────────────────────────────────────────────────┘
```

Panel: Picks (evidence→belief→recommendation trace per symbol)

```
┌──────────────────────────────────────────────────────────────┐
│ PICKS                                                        │
├──────────────────────────────────────────────────────────────┤
│ Symbol: 600000.SH   Action: BUY   Conviction: HIGH           │
│ Belief change:  b_t → b_{t+1}  | gain=0.42  decay=0.02       │
│                                                          ▸   │
│ Top Evidence (attention weights):                            │
│  (0.31) SentimentGold: net_sent +0.42, velocity +0.18        │
│  (0.22) MarketEvent: policy_positive mag 0.7                 │
│  (0.14) Influence: credible source, cross_confirm_1h high    │
│  (0.10) FundFlow: large_order_net_ratio rising               │
│  (0.07) Risk: regulatory_tone tightening penalty             │
│                                                          ▾   │
│ Calibration: predicted risk 0.12 (bin 0.1-0.2)  actual 0.09  │
```

```
│ Trace ID: ...  Model versions: kg_return_5d@...                │
└────────────────────────────────────────────────────────────────┘
```

Panel: Pipeline (execution + data-time + lineage + replay)

```
┌────────────────────────────────────────────────────────────────┐
│ PIPELINE                                                       │
├────────────────────────────────────────────────────────────────┤
│ DAG HEALTH:  ok=17  running=0  error=1                         │
│                                                          [↻]   │
│ [fetch] gate.evening  sentiment_fetch     OK  exec 22:01       │
│         data: bronze up to 2026-03-17  lag 1d                  │
│         lineage: rss/gdelt -> bronze                            │
│ [fetch] sentiment.fetched  sentiment_silver  OK  exec 22:03    │
│ [fetch] sentiment.silver_done sentiment_gold OK exec 22:08     │
│ [compute] sentiment.gold_done  event_extract   ERROR ...       │
│     └─ click: show error, replay upstream/full, backfill range │
└────────────────────────────────────────────────────────────────┘
```

Metrics, traceability, and alerts

EBRT's Trust contract requires:

- **Readiness gate**: operational vs research readiness, similar to the repo's `evaluate_gate` logic. [25]
- **Calibration**: Brier score and binning are already implemented for risk evaluation; EBRT makes it first-class and surfaced to UI. [25] [37]
- **Drift**: compute MMD between current evidence distribution and baseline; alert when drift exceeds threshold. [12]
- **Lineage**: every recommendation references top evidence IDs and belief transition IDs (`RecommendationTrace`).

This corresponds strongly with ML production readiness guidance: comprehensive testing/monitoring and data validation reduce "hidden technical debt". [9] Model Cards influence how you document active models and their evaluation slices. [10]

Evaluation plan

I recommend an evaluation stack matching the repo's daily evaluation philosophy:

1. **Unit tests**: deterministic extractors (`base_factors` noise score), aggregation metrics, attention weight normalisation, belief update invariants (weights sum to 1; bounded gains; monotonic confidence updates). [18]
2. **Replay tests**: re-run past dates with fixed evidence snapshots; verify reproducibility via fingerprints (the repo already computes cache fingerprints). [25]
3. **Backtests/event studies**: for each evidence type, estimate abnormal returns and hit rates, building "evidence IC" and "belief calibration". [6]
4. **A/B tests** (if you ever deploy live): compare baseline ranking vs EBRT belief updates for stability and drawdown.

5. **Calibration dashboards**: Brier, reliability bins, and per-sector calibration slices. [11]
6. **Drift monitoring**: MMD-based alarms and "belief inertia increase" when drift is high. [12]

Rollout timeline and Gantt plan

A practical rollout (no specific constraint on team/compute) is:

```
timeline
  title EBRT One-shot Refactor Rollout
  2026-03-18 : Baseline freeze (tag current behaviour)
  2026-03-19 : Introduce trade_py/engine API (thin wrappers)
  2026-03-22 : Migrate jobs to call engine (remove CLI calls)
  2026-03-26 : Add BeliefState + AttentionScore + BeliefTransition tables
  2026-03-30 : Produce recommendations via EBRT; keep legacy signals view
  2026-04-03 : Add RecommendationTrace + UI traces in Picks panel
  2026-04-07 : Trust hardening: calibration UI + drift alerts + run lineage
  2026-04-14 : Remove deprecated commands; final schema consolidation
```

```
gantt
  title EBRT Refactor Gantt
  dateFormat  YYYY-MM-DD
  section Foundations
  Freeze + golden tests               :a1, 2026-03-18, 2d
  Engine API skeleton                 :a2, 2026-03-19, 4d
  section Execution plane cleanup
  Jobs call engine (no CLI calls)     :b1, 2026-03-22, 5d
  DAG/RunRecord consolidation         :b2, 2026-03-24, 6d
  section Belief layer
  BeliefState + attention + residual  :c1, 2026-03-26, 6d
  BeliefTransition + trace            :c2, 2026-03-30, 5d
  section Decision and UI
  Recommendation pipeline + delta     :d1, 2026-04-01, 6d
  UI Today/Picks/Pipeline trace UX    :d2, 2026-04-03, 7d
  section Trust hardening
  Calibration + drift + alerts        :e1, 2026-04-07, 7d
```

Closing synthesis

The repository already implements much of EBRT's Evidence and Trust layers: evidence is captured in a structured Bronze/Silver/Gold pipeline with explicit noise and confidence metrics, and trust is surfaced via data health endpoints, runtime DAG status, and an unusually comprehensive daily evaluation gate with calibration metrics. [18] [17] [25] The missing middle is a first-class Belief layer and a cleaner execution plane, which also addresses the "too many commands / DAG calls commands / too many tables" pain. [1] [23]

EBRT's core algorithm is not "must be NN". Attention is a weighting mechanism; residual updates are stability tools; both can be implemented symbolically first (easy to validate, explain, observe) and later swapped to learned forms with backprop if data scale and evaluation maturity justify it. This is consistent with the research trajectory from attention (Transformers) to residual stability (ResNet) and

from belief revision principles (AGM) to filtering (Kalman) and MLOps trust practices (ML Test Score, Model Cards, calibration). [38]

---

[1] main.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/cli/main.py

[2] https://www.cambridge.org/core/services/aop-cambridge-core/content/view/7ED837BAD5FB6D9A7C77906D73527F9C/S0022481200032849a.pdf/on-the-logic-of-theory-change-partial-meet-contraction-and-revision-functions.pdf
https://www.cambridge.org/core/services/aop-cambridge-core/content/view/7ED837BAD5FB6D9A7C77906D73527F9C/S0022481200032849a.pdf/on-the-logic-of-theory-change-partial-meet-contraction-and-revision-functions.pdf

[3] [26] https://www.bibsonomy.org/bibtex/46000cea585242435901ab4a9f5738b5
https://www.bibsonomy.org/bibtex/46000cea585242435901ab4a9f5738b5

[4] [35] [38] https://data.inu.ac.kr/items/78e9c3f4-387d-4b07-8e79-3650b8a5117b
https://data.inu.ac.kr/items/78e9c3f4-387d-4b07-8e79-3650b8a5117b

[5] https://www.microsoft.com/en-us/research/publication/deep-residual-learning-for-image-recognition/
https://www.microsoft.com/en-us/research/publication/deep-residual-learning-for-image-recognition/

[6] https://www.sciencedirect.com/science/article/pii/0304405X8590042X
https://www.sciencedirect.com/science/article/pii/0304405X8590042X

[7] https://dokk.org/library/theoryofcomputing_v011a004
https://dokk.org/library/theoryofcomputing_v011a004

[8] https://www.jmlr.org/papers/v11/vincent10a.html
https://www.jmlr.org/papers/v11/vincent10a.html

[9] https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/
https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/

[10] [36] https://research.google/pubs/model-cards-for-model-reporting/
https://research.google/pubs/model-cards-for-model-reporting/

[11] [37] https://ui.adsabs.harvard.edu/abs/1950MWRv...78....1B/abstract
https://ui.adsabs.harvard.edu/abs/1950MWRv...78....1B/abstract

[12] https://www.jmlr.org/beta/papers/v13/gretton12a.html
https://www.jmlr.org/beta/papers/v13/gretton12a.html

[13] README.md
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/README.md

[14] claude-latest-plan.md
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/docs/claude-latest-plan.md

[18] base_factors.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/intelligence/base_factors.py

[19] enricher.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/intelligence/enricher.py

[17] aggregate.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/data/pipeline/aggregate.py

20 event.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/cli/event.py

21 run.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/cli/run.py

22 __init__.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/bus/__init__.py

23 __init__.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/jobs/__init__.py

24 app.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_web/backend/app.py

25 service.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/evaluation/service.py

27 pipeline_db.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/db/pipeline_db.py

28 feed_scorer.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/intelligence/feed_scorer.py

29 web-ui-v3-plan.md
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/docs/web-ui-v3-plan.md

30 https://lightgbm.readthedocs.io/en/v4.1.0/_modules/lightgbm/engine.html
https://lightgbm.readthedocs.io/en/v4.1.0/_modules/lightgbm/engine.html

31 gateway.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/data/access/gateway.py

32 window_scorer.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/signals/window_scorer.py

33 inference.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_web/backend/inference.py

34 migrations.py
https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/db/migrations.py